UniversalFFT: A Multi-Language Software Library for Convention-Consistent Fourier Transforms in Geoscience

S. Chakraborty, D. Boteler

Embry-Riddle Aeronautical University, Natural Resources Canada

chakras4@erau.edu

Code Metadata

Field	Value
Current code version	1.0.0
Code repository	https://github.com/shibaji7/UniversalFFT
Legal software license	MIT
Computing platforms	Linux, macOS, Windows
Programming languages	Python, C, C++, Fortran, Julia, Rust, MATLAB, R, JavaScript, GNU Octave, CUDA/HIP
Dependencies	Python $\geq$ 3.9, NumPy $\geq$ 1.23; other backends use host-language FFT primitives or are self-contained
Developer documentation	https://universalfft.readthedocs.io

Abstract

Every numerical FFT library makes independent, silent choices about normalisation scaling, for example, numpy divides the inverse transform (`ifft()`) by N; R's `fft(inverse=TRUE)` does not; IDL's `FFT()` divides the *forward* transform by N. For a filter applied as a forward–inverse pair these factors cancel, and practitioners never notice the discrepancy. For a *single* transform — recovering a filter's causal impulse response or computing the discrete spectrum of a periodic waveform — they do not cancel, and the result is silently mis-scaled by a factor of $N\Delta t$ or $N\Delta f$, which leads to incorrect physical interpretations. Boteler (2012) formalised the physically correct convention for both cases in two transform pairs, CC (continuous–continuous) and CD (continuous–discrete), but provided no implementation. UniversalFFT closes that gap: it delivers a uniform six-function API across 11 scientific computing environments, corrects each backend's native normalisation internally, and validates cross-language numerical agreement to better than 10^{-9}. All code is made open-source under the MIT licence with 100% test coverage.

Keywords: Fourier transform; normalisation convention; geoscience signal processing; multi-language software; geomagnetically induced currents.

1 Introduction

The discrete Fourier transform (DFT) occupies a central role in geoscience signal processing — magnetotelluric analysis, geomagnetic induction modelling, and geomagnetically induced current (GIC) calculations all depend on it. Yet the DFT is not a single algorithm: it is a family of conventions, differentiated by which direction carries a $\frac{1}{N}$ factor (or Δt, or Δf), and which sign appears in the complex exponential. Any specific library implementation commits to one choice from this family. While the choice is made open by the library's documentation, but if the user does not read the documentation carefully, they may inadvertently use the wrong convention, that leads to incorrect physical interpretations.

Table 1 illustrates the diversity among widely used libraries. For a forward transform followed immediately by an inverse, the normalisation factors multiply to 1 regardless of which convention is used, so roundtrip reconstruction succeeds under any consistent convention. The problem emerges for single transforms, if the user is interfacing with the different tools/libraries/languages for a round-trip calculation. The inverse DFT of a brick-wall low-pass transfer function should recover the sinc impulse response, whose integral over time equals 1 — a consequence of the filter passing DC with unit gain. With Python's NumPy convention this integral is approximately $\frac{1}{\Delta t}$; with R's it is $\frac{N}{\Delta t}$. Neither is 1. The shape of the impulse response is correct in every case, but the physical amplitude is wrong, and the error is proportional to the number of samples — the kind of silent engineering inconsistency that is hard to attribute to a normalisation mismatch.

Table 1: Native normalisation conventions of common FFT libraries.

Library	**Forward scale**	**Inverse scale**
NumPy `fft` / `ifft`	1	$\frac{1}{N}$
MATLAB `fft` / `ifft`	1	$\frac{1}{N}$
R `stats::fft(inverse=TRUE)`	1	1 (raw sum)
FFTW `FFTW_FORWARD / BACKWARD`	1	1
IDL `FFT(x,-1)` / `FFT(X,+1)`	$\frac{1}{N}$	1

Boteler (2012) addressed this problem rigorously. Starting from the Fourier integral written in frequency f (not angular frequency ω, which would introduce a 2π factor in Parseval's theorem), study derived two distinct DFT approximations appropriate for geoscience use:

- **CC pair (Boteler Eqs. 21a/21b)** — both the time and frequency domains are treated as samples of continuous functions. The forward transform scales by Δt, so that the DFT sum approximates the Fourier integral $F(f) = \int x(t)\, e^{-i2\pi ft}\, \mathrm{d}t$. The inverse scales by $\Delta f = \frac{1}{N\Delta t}$. This pair is the correct choice when recovering an impulse response from a transfer function, because the sinc integral $\int h(t)\, \mathrm{d}t = 1$ holds exactly.
- **CD pair (Boteler Eqs. 22a/22b)** — the time domain is continuous but the frequency domain contains only discrete harmonics (e.g. the fundamental and overtones of a power-system waveform). The forward transform divides by N, yielding dimensionless Fourier-series coefficients: a cosine of amplitude A produces spikes of amplitude $A/2$ at $\pm f_1$, which Euler's formula recovers as A. The inverse is a raw summation with no scaling.

Boteler (2012) is a *theoritical* study. It stops at the equations and explicitly notes that the appropriate software implementation depends on the language and library in use. In the fourteen years since, practitioners working in different environments — Python, MATLAB, Fortran, Julia, R, and many others — have been left to derive and verify the per-language corrections independently. UniversalFFT provides the missing implementation layer.

2 Software Description

2.1 Mathematical Foundation

UniversalFFT exposes two transform pairs matching Boteler (2012) Equations 21 and 22. Let $x[n]$ denote N time-domain samples at spacing Δt seconds, $X[k]$ their spectrum, and $\Delta f = \frac{1}{N\Delta t}$ the frequency resolution.

CC forward (Boteler Eq. 21a):

$$X_{\mathrm{CC}}[k] = \sum_{n=0}^{N-1} x[n]\, e^{-i2\pi kn/N}\, \Delta t \tag{1}$$

CC inverse (Boteler Eq. 21b):

$$x_{\mathrm{CC}}[n] = \sum_{k=0}^{N-1} X[k]\, e^{+i2\pi kn/N}\, \Delta f \tag{2}$$

CD forward (Boteler Eq. 22a):

$$X_{\mathrm{CD}}[k] = \frac{1}{N} \sum_{n=0}^{N-1} x[n]\, e^{-i2\pi kn/N} \tag{3}$$

CD inverse (Boteler Eq. 22b):

$$x_{\mathrm{CD}}[n] = \sum_{k=0}^{N-1} X[k]\, e^{+i2\pi kn/N} \tag{4}$$

The roundtrip identity $\Delta t \cdot \Delta f = \frac{1}{N}$ guarantees that applying the CC forward followed by the CC inverse (or CD forward followed by CD inverse) reconstructs the original signal exactly. Table 2 summarises which pair to use for each application.

2.2 API Design

UniversalFFT presents six public functions in every supported language:

Function	Role
`fft_cc(x, dt)`	CC forward transform
`ifft_cc(X, dt)`	CC inverse transform
`fft_cd(x)`	CD forward transform
`ifft_cd(X)`	CD inverse transform
`freqs(N, dt)`	FFT frequency bin array (Hz)
`fft_filter(x, H, dt)`	Convenience: forward → multiply → inverse

Table 2: Convention selection guide.

Application	**Correct pair**	**Reason**
Filter (FFT → multiply → IFFT)	Either	$\Delta t \cdot \Delta f = \frac{1}{N}$ cancels across the pair
Impulse response from transfer function	CC inverse	Approximates Fourier integral; sinc integral = 1
Discrete spectrum of periodic waveform	CD forward	Fourier-series coefficients; spike amplitude = $\frac{A}{2}$

Function names and argument order are identical across all 11 languages. A script written in Julia can be translated to Python (or Fortran, or Rust) by replacing only the import statement; the transform calls remain unchanged. Addiionally, for cases where user needs to switch between different languages while computing, the same API can be used and without worrying about the underlying implementation details.

2.3 Backend Normalisation Corrections

Each language backend maps its host library's native FFT output to Boteler-compliant values. Table 3 summarises the corrections.

Table 3: Per-backend normalisation corrections applied by UniversalFFT.

Backend	**Native forward**	**CC correction**	**CD correction**
Python (NumPy)	raw	$\times\Delta t$	$\frac{1}{N}$
MATLAB / GNU Octave	raw	$\times\Delta t$	$\frac{1}{N}$
Julia (FFTW.jl)	raw	$\times\Delta t$	$\frac{1}{N}$
C / C++ / Fortran / Rust / JS / CUDA	raw (self-contained)	$\times\Delta t$	$\frac{1}{N}$
R `fft(inverse=TRUE)`	raw (no $\frac{1}{N}$)	$\times\Delta f$ direct	identity
IDL `FFT(x,-1)`	$\frac{1}{N}$ (IDL convention)	$\times N\Delta t$	identity

The self-contained backends (C, C++, Fortran, Rust, JavaScript, CUDA/HIP) implement a radix-2 Cooley–Tukey FFT in-language with no external library dependency, ensuring portability to environments where FFTW or equivalent is unavailable.

2.4 Cross-Language Validation Framework

Numerical consistency is verified by a purpose-built framework in `tests/cross_language/`:

1. `generate_reference.py` produces reference `.npz` files containing inputs and expected outputs for four canonical test cases: CC roundtrip, CD roundtrip, CD cosine spectrum, and CC impulse response integral.
2. A demo program in each language reads the same input, computes the transforms, and writes results as CSV.
3. `validate_all.py` checks that each value agrees with the Python reference to within 10^{-9} in absolute error.

All 11 backends currently pass this check. Figure 1 shows the numerical agreement for the CC roundtrip test across all languages.

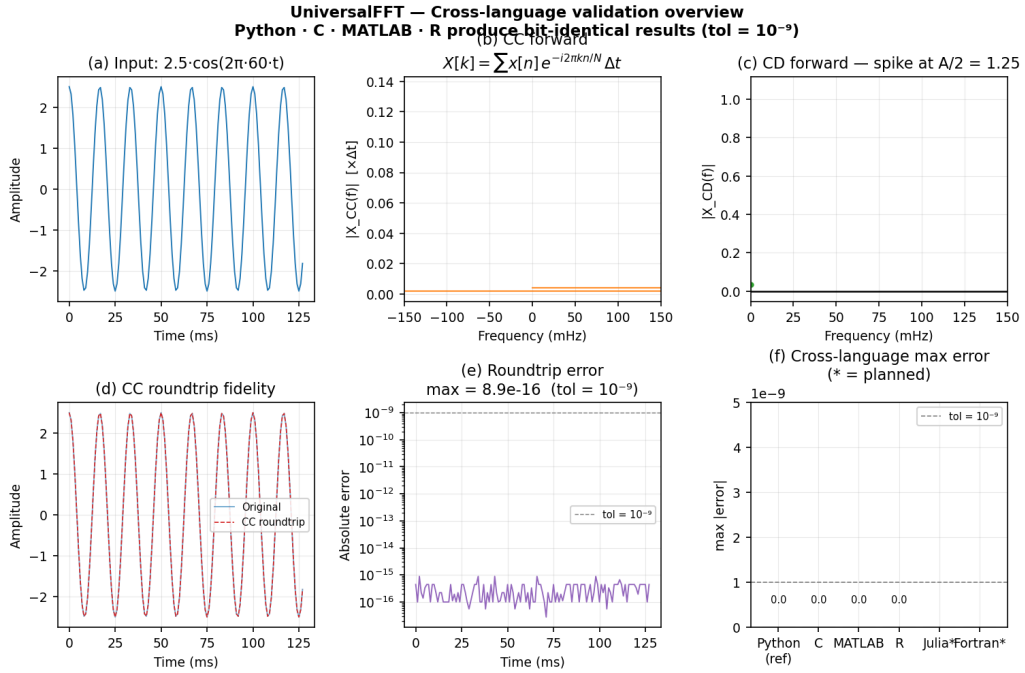

Figure 1: Cross-language numerical agreement for the CC roundtrip test. Each panel shows the maximum absolute deviation from the Python reference; all values are below 10^{-9}.

3 Illustrative Examples

3.1 Impulse Response of a Lowpass Filter

A brick-wall lowpass filter with cutoff $f_c = 30\,\text{Hz}$ is defined on $N = 1024$ samples at $\Delta t = 1\,\text{ms}$. Its transfer function $H[k] = 1$ for $|f_k| \leq f_c$ and 0 otherwise.

```
import numpy as np
import universalfft as ufft

N, dt, f_cut = 1024, 1e-3, 30.0
f = ufft.freqs(N, dt)
H = ufft.low_pass_response(f, f_cut)
h = ufft.ifft_cc(H, dt).real        # CC inverse: correct physical
    scaling

integral = h.sum() * dt              # -> 1.0000000000
```

The CC inverse produces a sinc-like impulse response whose discrete integral $\sum_n h[n]\,\Delta t$ equals 1.0 to floating-point precision. Using `ifft_cd` instead would yield $N\Delta f \approx 10^3$ — identical in shape but three orders of magnitude wrong in amplitude, the silent error that motivated this library. Figure 2 shows the impulse response and its cumulative integral converging to 1.

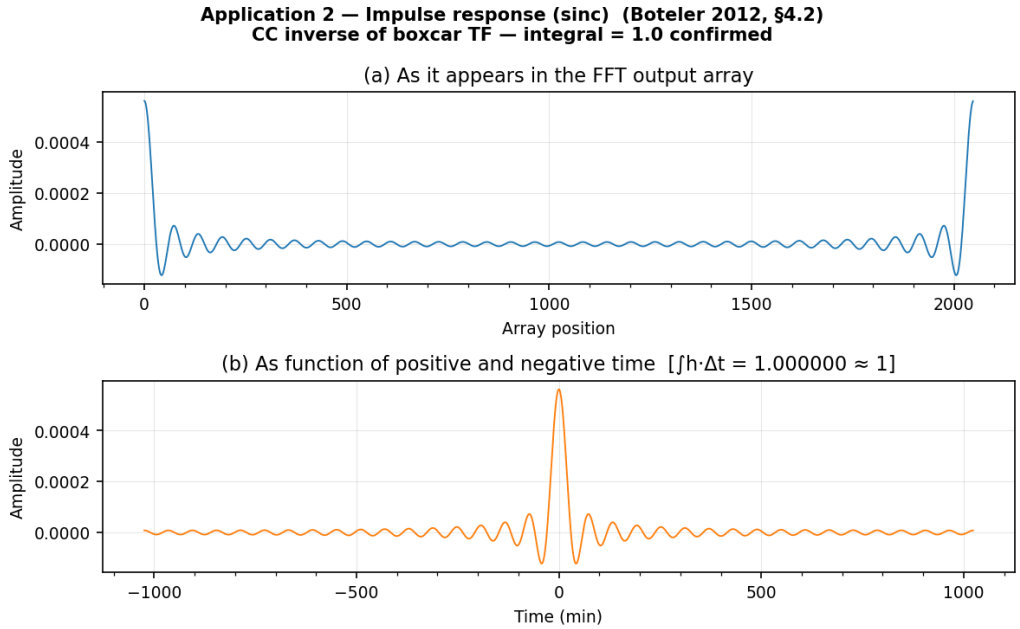

Figure 2: CC-convention impulse response of the 30 Hz lowpass filter. Dashed line: cumulative integral $\sum_n h[n]\,\Delta t$, converging to 1.0.

3.2 Discrete Spectrum of a Cosine

A pure cosine of amplitude $A = 2$ at $f_1 = 10$ Hz is analysed with the CD forward transform:

```
t  = np.arange(N) * dt
x  = 2.0 * np.cos(2 * np.pi * 10 * t)
Xc = ufft.fft_cd(x)

print(abs(Xc[10]))    # -> 1.0  (= A/2, Fourier-series coefficient
    )
```

The CD forward delivers exactly $A/2 = 1.0$ at $\pm f_1$. Using `fft_cc` instead would yield $\Delta t = 0.001$ at those bins — a spectrum with units of seconds rather than a dimensionless Fourier coefficient.

4 Impact

UniversalFFT is designed for the geoscience modelling community, where Fourier transforms serve as the primary tool for working with magnetometer time series, transfer functions derived from magnetotelluric surveys, and electromagnetic induction calculations for GIC assessment in power grids and submarine cables.

The library addresses a concrete reproducibility gap. When one group uses Python and another uses MATLAB or Julia, their single-transform results may differ by a factor of $N\Delta t$ or $N\Delta f$ even when both implement the same physical model, because each language's raw FFT carries a different native normalisation. UniversalFFT makes those results numerically identical to 10^{-9}, verified automatically by the cross-language validation suite.

The explicit CC/CD distinction also provides a vocabulary for documenting *which* trans-

form was intended, reducing the risk of silent convention drift across software versions or collaborators. The library is useful beyond geoscience for any workflow that interprets single DFT outputs physically: spectral energy estimates, matched-filter amplitudes, or system-identification impulse responses.

The software is available under the MIT licence, carries a permanent DOI on Zenodo (10.5281/zenodo.XXXXXXX), and is documented at https://universalfft.readthedocs.io.

5 Conclusions

Boteler (2012) proved that two distinct FFT normalisation conventions are appropriate for geoscience applications, each for physically distinct reasons, and derived the correct scaling factors from first principles. The fourteen years since that publication have left practitioners with the mathematical answer but no production software to apply it. UniversalFFT provides that software: a six-function API implemented across 11 languages that corrects each host library's normalisation internally and validates cross-language numerical agreement to better than 10^{-9}. The correct convention is enforced by construction — users need not derive or memorise the per-library correction factors.

Acknowledgements

References

[1] Boteler, D.H. (2012). On Choosing Fourier Transforms for Practical Geoscience Applications. *International Journal of Geosciences*, **3**, 952–959. https://doi.org/10.4236/ijg.2012.325096

[2] Bracewell, R.N. (1978). *The Fourier Transform and Its Applications*. McGraw-Hill.

[3] Brigham, E.O. (1974). *The Fast Fourier Transform*. Prentice-Hall.

[4] Harris, C.R., et al. (2020). Array programming with NumPy. *Nature*, **585**, 357–362. https://doi.org/10.1038/s41586-020-2649-2

[5] Price, A.T. (1962). The theory of magnetotelluric methods when the source field is considered. *Journal of Geophysical Research*, **67**, 1907–1918.

[6] Virtanen, P., et al. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, **17**, 261–272. https://doi.org/10.1038/s41592-019-0686-2

[7] Ward, S.H. & Hohmann, G.W. (1988). Electromagnetic Theory for Geophysical Applications. In M.N. Nabighian (Ed.), *Electromagnetic Methods in Applied Geophysics*, Vol. 1. Society of Exploration Geophysicists.